

FIT BUDDY

AI Fitness Plan Generator Using Gemini Models

Project Documentation

Project Type	Web Application
Frontend	HTML5, CSS3
Backend	Python Flask
Database & AI	SQLite + Google Gemini API

Prepared from the implemented Fit Buddy application and submitted UI screenshots

CERTIFICATE

This is to certify that the project entitled "Fit Buddy – AI Fitness Plan Generator Using Gemini Models" has been successfully developed as a five-member group project under Team ID PraveenaP1209/. The project demonstrates the integration of a Python Flask web application, SQLite database, fitness calculations, and Google Gemini AI for generating personalized general fitness guidance.

Team ID: PraveenaP1209/.

Team Size: 5

Guide: _____

Team Leader: Praveena P

Team Member: Sadhana D

Team Member: Yuvasri S

Team Member: Nivetha B

Team Member: Rudhrapriya M

DECLARATION

We hereby declare that the project “Fit Buddy – AI Fitness Plan Generator Using Gemini Models” has been developed by our five-member team as an academic software project. The application combines web development, database storage, fitness calculations, and generative AI to demonstrate a practical real-world use case.

Team ID: PraveenaP1209/

Team Size: 5

Guide: _____

Team Leader: : Praveena P

Team Member: Sadhana D

Team Member: Yuvasri S

Team Member: Nivetha B

Team Member: Rudhrapriya M

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to our project guide for providing valuable guidance, support, and encouragement throughout the development of our project “Fit Buddy – AI Fitness Plan Generator Using Gemini Models.”

We are thankful to the faculty members and the Department for providing us with the necessary resources, facilities, and support to complete this project successfully.

We would also like to thank our friends and classmates for their valuable suggestions, encouragement, and support during the development, testing, and documentation of this project.

We are grateful to everyone who directly or indirectly contributed to the successful completion of our project. Working together as a five-member team helped us gain practical knowledge and experience in Python, Flask, SQLite, HTML, CSS, JavaScript, and Google Gemini AI integration, while also improving our teamwork, communication, and problem-solving skills.

We sincerely thank everyone who supported us in completing this project successfully.

ABSTRACT

Fit Buddy is an AI-assisted fitness planning web application designed as a college-level project. The application collects a user's basic fitness profile such as age, gender, height, weight, activity level, fitness goal, experience, available workout days, equipment, and diet preference. Based on the profile, the application calculates basic fitness indicators such as BMI, BMR, and estimated maintenance calories. The application can then use Google Gemini to generate a structured seven-day general fitness plan.

The implemented interface includes a landing page, registration and login pages, a personalized dashboard, fitness profile management, AI-generated plan display, plan history, and an AI chat section. The AI chat allows the user to ask questions related to the current plan, such as breakfast choices, exercise alternatives, or recovery.

The project demonstrates how a Flask application can be combined with a generative AI service to create a personalized user experience while keeping the architecture understandable for a student project.

TABLE OF CONTENTS

1. Introduction
2. Project Description
3. Problem Statement
4. Existing System
5. Proposed System
6. Objectives
7. Scope of the Project
8. System Requirements
9. Core Technologies
10. Project Architecture
11. Project Workflow and Milestones
12. Module Description
13. Database Design
14. Gemini AI Integration
15. User Interface Design
16. Implementation and Project Structure
17. Testing
18. Error Handling and Troubleshooting
19. Advantages
20. Limitations
21. Future Enhancements
22. Conclusion
23. References
24. Appendix – Application Screenshots

1. INTRODUCTION

Fitness planning often requires users to understand their current condition, select realistic goals, choose suitable exercises, and maintain consistency. Fit Buddy simplifies this process by combining basic fitness calculations with generative AI. The project focuses on a clean student-friendly web application rather than a complex commercial fitness platform.

The system follows a simple flow: register or log in, enter fitness information, view calculated fitness values, generate an AI plan, review the seven-day schedule, and ask questions through the AI assistant.

2. PROJECT DESCRIPTION

Fit Buddy – AI Fitness Plan Generator Using Gemini Models is a Flask-based web application. It stores user and plan information using SQLite and uses Gemini for natural-language fitness plan generation and plan-related questions.

3. PROBLEM STATEMENT

Many users need a simple starting point for organizing workouts and healthy routines but may not know how to structure a weekly plan. The project addresses this by accepting profile details and generating a readable, personalized general fitness plan. The objective is educational and demonstrative; the system is not intended to replace a doctor, dietitian, or qualified fitness professional.

4. EXISTING SYSTEM

A basic approach is to search for workout plans manually or use fixed static schedules. Such approaches may not adapt to the user's goal, experience, equipment, or available workout days. A purely static plan also does not provide an interactive question-and-answer experience.

5. PROPOSED SYSTEM

The proposed Fit Buddy system combines structured profile information, fitness calculations, database storage, and Gemini-generated responses. The application provides both a generated plan and an interactive AI question area so that users can ask follow-up questions about the displayed plan.

6. OBJECTIVES

- Build a simple web-based fitness planning application.
- Calculate basic fitness values from user profile data.
- Integrate Google Gemini for personalized general fitness guidance.
- Provide a readable seven-day workout plan.
- Provide a plan-specific AI question interface.
- Store users and generated plans in SQLite.
- Demonstrate authentication, backend routing, database handling, and AI integration in one project.

7. SCOPE OF THE PROJECT

The current scope covers general fitness planning, profile management, basic calculations, AI-generated workout and nutrition guidance, plan history, and plan-related questions. The screenshots show a browser-based application with a dashboard, authentication pages, plan display, and AI chat.

The project does not establish medical diagnoses, prescribe medication, or provide guaranteed health or weight-loss outcomes.

8. SYSTEM REQUIREMENTS

Category	Requirement
Language	Python
Web Framework	Flask
Database	SQLite
AI Service	Google Gemini API
Frontend	HTML5, CSS3
Environment	Python virtual environment (.venv)
Browser	Modern web browser
Configuration	.env file for secrets/API configuration

9. CORE TECHNOLOGIES

Technology	Purpose
Python	Main programming language for backend logic and calculations.
Flask	Web framework for routes, authentication flow, rendering templates, and application logic.
SQLite	Lightweight database used for local application data.
SQLAlchemy	ORM layer for database model and persistence operations.
Flask-Login	Supports user sessions and protected routes.
Bootstrap	Responsive UI layout and components.
Google Gemini	Generates fitness plans and answers plan-related questions.

10. PROJECT ARCHITECTURE

The application follows a simple layered architecture. The browser communicates with Flask routes. Flask processes authentication, profile information, fitness calculations, database operations, and Gemini requests. Generated plans are stored and later displayed through Jinja templates.

- Presentation Layer – HTML templates, CSS.
- Application Layer – Flask routes and business logic.
- Calculation Layer – BMI, BMR, and maintenance calorie calculations.
- Data Layer – SQLite database and SQLAlchemy models.
- AI Layer – Gemini client and prompt construction.

11. PROJECT WORKFLOW AND MILESTONES

Milestone 1 – Gemini AI Initialization

Configure Gemini API access, environment variables, model selection, and basic response testing.

Milestone 2 – Core Functionality Development

Implement user profile data, BMI/BMR/maintenance calculations, plan generation, and plan storage.

Milestone 3 – Flask Backend Integration

Implement authentication, protected routes, database models, plan routes, and the AI question endpoint.

Milestone 4 – UI Development

Create the landing page, login/register screens, dashboard, plan page, AI chat, and responsive styling.

Milestone 5 – Testing and Optimization

Test form flows, plan generation, database behavior, AI response handling, error handling, and UI presentation.

12. MODULE DESCRIPTION

Module	Function
Home Module	Introduces Fit Buddy and explains the simple three-step process.
Authentication Module	Provides registration, login, logout, and protected user access.
Profile Module	Stores fitness details used by calculations and the AI layer.
Calculator Module	Calculates BMI, BMR, and estimated maintenance calories.
Dashboard Module	Shows fitness metrics, current goal, profile summary, and previous plans.
Plan Generation Module	Builds a prompt from the profile and requests a seven-day plan from Gemini.
Plan Display Module	Presents generated content with structured headings and readable sections.
AI Chat Module	Allows questions related to the current fitness plan.
Plan History Module	Lists previously generated plans for the logged-in user.

13. DATABASE DESIGN

The project uses SQLite for local persistence. The implemented application structure supports user accounts, fitness profile information, and generated fitness plans so that the dashboard can display the current profile and plan history.

User: Stores identity, login information, and role information.

FitnessProfile: Stores age, height, weight, goal, experience, activity, equipment, and diet.

FitnessPlan: Stores generated plan content and its relationship to the user.

14. GEMINI AI INTEGRATION

The AI layer receives structured profile information and constructs a prompt requesting a practical seven-day general fitness plan. The application can also send the current plan together with a user's question so that Gemini can answer in the context of that plan.

1. User opens a generated plan.
2. User enters a question such as "What should I eat for breakfast?"
3. Browser sends the question to the Flask plan-question endpoint.
4. Flask verifies access to the selected plan.
5. The plan content and question are passed to Gemini.
6. Gemini returns an answer.
7. The browser displays the response in the AI chat bubble.

The AI prompt is designed to provide general fitness guidance and to avoid medical diagnosis, medication advice, starvation, extreme dieting, and guaranteed weight-loss claims.

15. USER INTERFACE DESIGN

The submitted screenshots show a clean, soft, student-friendly visual style. The application uses a light background, rounded cards, purple accent elements, compact navigation, and clear section headings.

- Home page with project introduction and three-step workflow.
- Login and registration cards centered on the page.
- Dashboard cards for BMI, BMR, and maintenance calories.
- Current goal card with experience, activity, diet, and equipment.
- AI-generated plan page with Gemini badge and print option.
- AI chat panel with online indicator, user/AI bubbles, and suggested questions.

16. IMPLEMENTATION AND PROJECT STRUCTURE

The submitted project structure contains a Flask application package, templates, static CSS, service code, database instance, environment configuration, dependency file, and a run script.

```
FitBuddy Project
├── .venv/
├── app/
│   ├── services/
│   │   └── gemini_service.py
│   ├── static/css/
│   │   └── style.css
│   └── templates/
│       ├── admin.html
│       ├── base.html
│       ├── dashboard.html
│       ├── home.html
│       ├── login.html
│       ├── plan.html
│       ├── profile.html
│       └── register.html
└── __init__.py
```

```

├── auth.py
├── calculator.py
├── extensions.py
├── main.py
├── models.py
├── instance/fitbuddy.db
├── .env
├── .env.example
├── .gitignore
├── PROJECT_REPORT.md
├── README.md
├── requirements.txt
└── run.py

```

17. TESTING

ID	Test	Expected Result	Status
TC-01	Open home page	Home page loads with project introduction and navigation	Pass
TC-02	Register new user	Registration accepts valid user details	Pass
TC-03	Login	Valid credentials open dashboard	Pass
TC-04	View dashboard	BMI, BMR, maintenance and current goal are displayed	Pass
TC-05	Generate plan	Plan generation creates a plan record and opens plan page	Implemented / API dependent
TC-06	View plan history	Previous plans appear in dashboard history	Pass
TC-07	Ask AI question	Question is answered when Gemini authentication succeeds	API dependent
TC-08	Invalid AI credentials	Application logs a controlled Gemini failure	Observed

18. ERROR HANDLING AND TROUBLESHOOTING

The submitted development screenshot shows a Gemini authentication error. The Flask log reports HTTP 401 UNAUTHENTICATED with ACCESS_TOKEN_TYPE_UNSUPPORTED. The same log confirms that GEMINI_API_KEY was loaded and that the selected model was gemini-2.5-flash. This isolates the observed failure to Gemini authentication/configuration rather than a missing environment variable.

The application also logs an SDK warning recommending chat-based sending for automatic function calling. This warning is separate from the 401 authentication failure.

8. Create or use a currently supported Gemini API key in Google AI Studio.
9. Keep GEMINI_API_KEY in the project .env file and never commit the real key.
10. Check for an unintended GOOGLE_API_KEY environment variable that may override GEMINI_API_KEY.
11. Restart the terminal and Flask application after changing environment variables.

12. Run a standalone Gemini test before testing the Flask AI chat.

19. ADVANTAGES

- Simple and understandable college-level architecture.
- Personalized plan generation instead of a completely fixed plan.
- Interactive AI question-and-answer feature.
- Local SQLite storage keeps setup lightweight.
- Responsive and clean user interface.
- Modular Flask structure separates authentication, calculations, models, and Gemini service code.

20. LIMITATIONS

- Gemini functionality depends on valid external API authentication and network access.
- Generated fitness guidance is general and should not be treated as medical advice.
- The application does not replace professional coaching or medical assessment.
- SQLite is suitable for a student/local project but may not be ideal for large-scale multi-user deployment.
- The AI chat depends on the server providing the plan context with the question.

21. FUTURE ENHANCEMENTS

- Workout completion tracking and progress charts.
- Weekly weight and measurement tracking.
- Persistent AI conversation history.
- Exercise images or demonstrations.
- Mobile-friendly PWA or mobile application.
- Tamil and other multilingual AI responses.
- Configurable reminder notifications.
- PostgreSQL for larger deployments.

22. CONCLUSION

Fit Buddy demonstrates the integration of a Flask web application, local database storage, fitness calculations, responsive user interface, and generative AI. The submitted screenshots show the user flow from landing page and authentication through dashboard, AI-generated plan display, and plan-specific AI chat.

The project provides an academic demonstration of how an AI API can be integrated into a conventional web application. The observed Gemini authentication issue is isolated from the application's core UI and database functionality.

23. REFERENCES

- Google Gemini API documentation.
- Python documentation.

- Flask documentation.
- SQLAlchemy documentation.
- Bootstrap documentation.
- Google AI Studio – Gemini API key management.

24. APPENDIX – APPLICATION SCREENSHOTS

Figure 1. Fit Buddy Home Page

Landing page introducing Fit Buddy and the three-step fitness planning workflow.

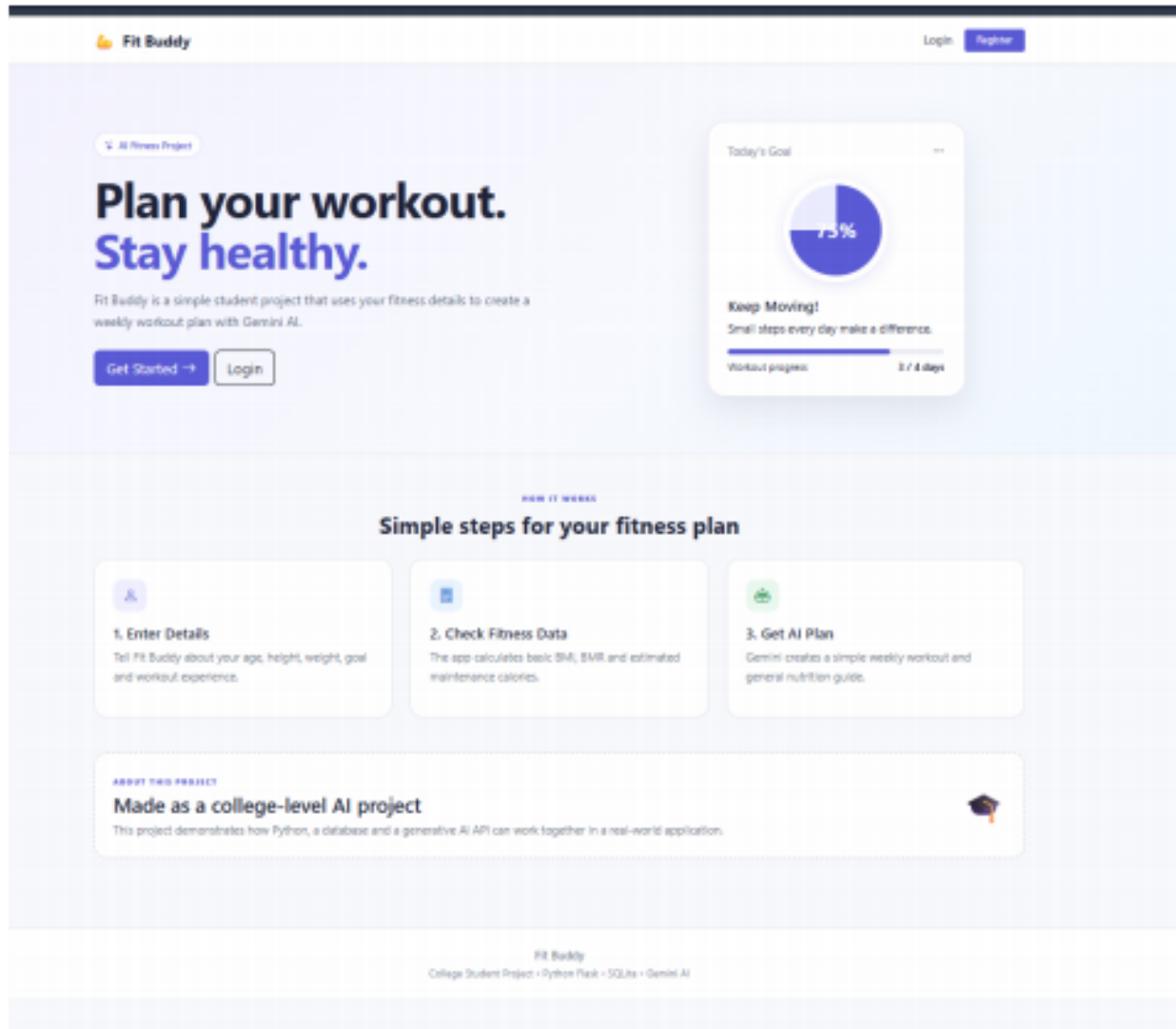


Figure 1. Fit Buddy Home Page

Figure 2. Login Page

User login page for accessing the personalized fitness dashboard.

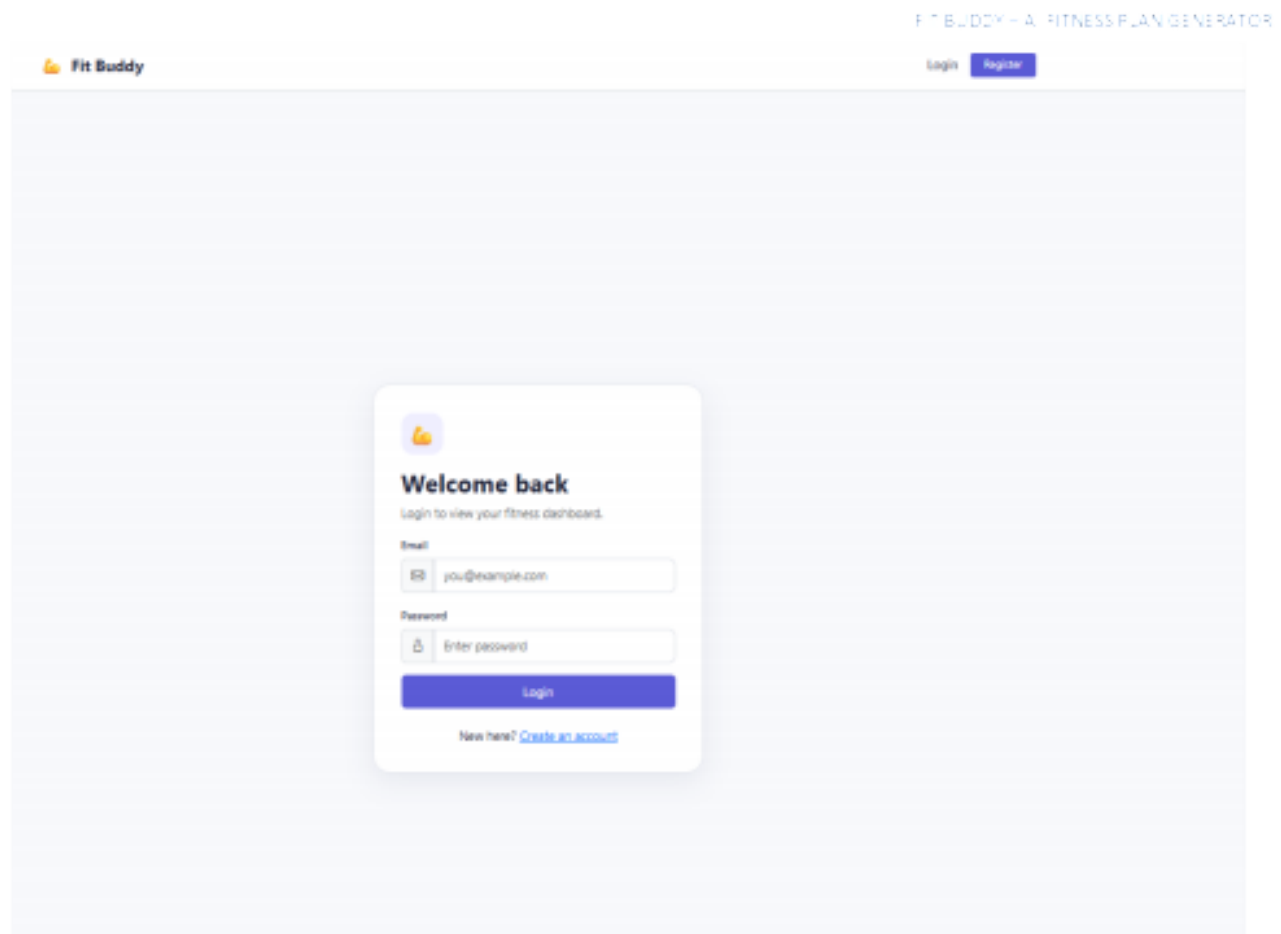


Figure 2. Login Page

Figure 3. Registration Page

New user registration page for creating a Fit Buddy account.

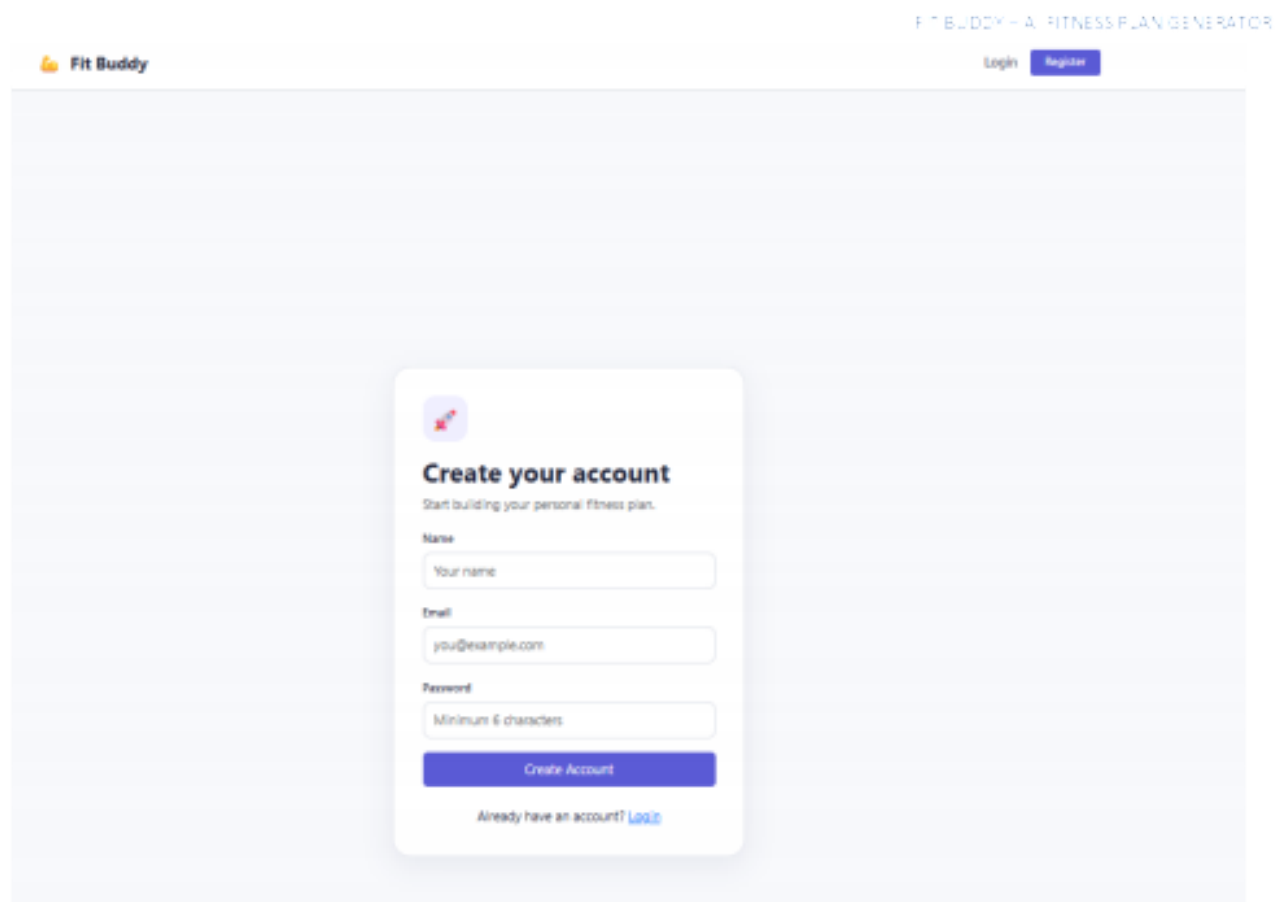


Figure 3. Registration Page

Figure 4. My Fitness Profile

Fitness Profile page where the user enters basic information, fitness goal, workout experience, workout days, diet preference, available equipment, and saves the profile. The authenticated navigation also includes Dashboard, My Profile, and Logout.

 Fit Buddy

DashboardMy ProfileLogout

STEP 1

My Fitness Profile

Enter your details and we'll use them to prepare your plan.

Basic Information

Age

Gender

Height (cm)

Weight (kg)

Activity Level

Male

170

85

Sedentary

Your Fitness Goal

Goal

Workout Experience

Workout Days / Week

Diet Preference

General Fitness

Beginner

1

No Preference

Available Equipment

☐ No Equipment

☐ Dumbbells

☐ Resistance Band

☐ Barbell

☐ Gym Machines

Tip: Select only the equipment you can actually use at home or in your gym.

Save My Profile ✓

Figure 4. My Fitness Profile

Figure 5. User Dashboard

Dashboard showing BMI, estimated BMR, maintenance calories, current goal, fitness profile summary, AI plan generation, and plan history.

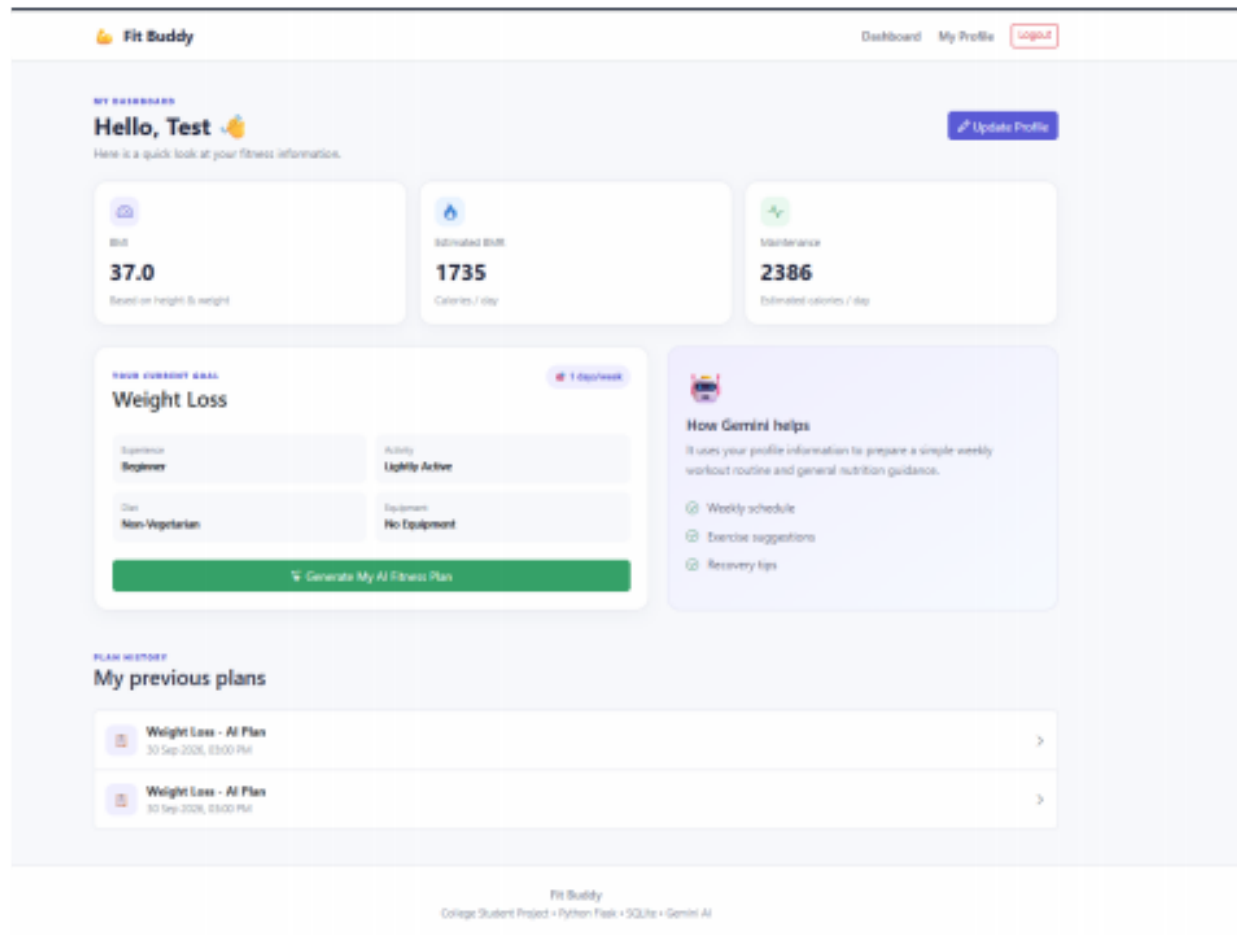


Figure 5. User Dashboard

Figure 6. AI Generated Fitness Plan

Generated fitness plan page showing the goal summary and personalized profile information.

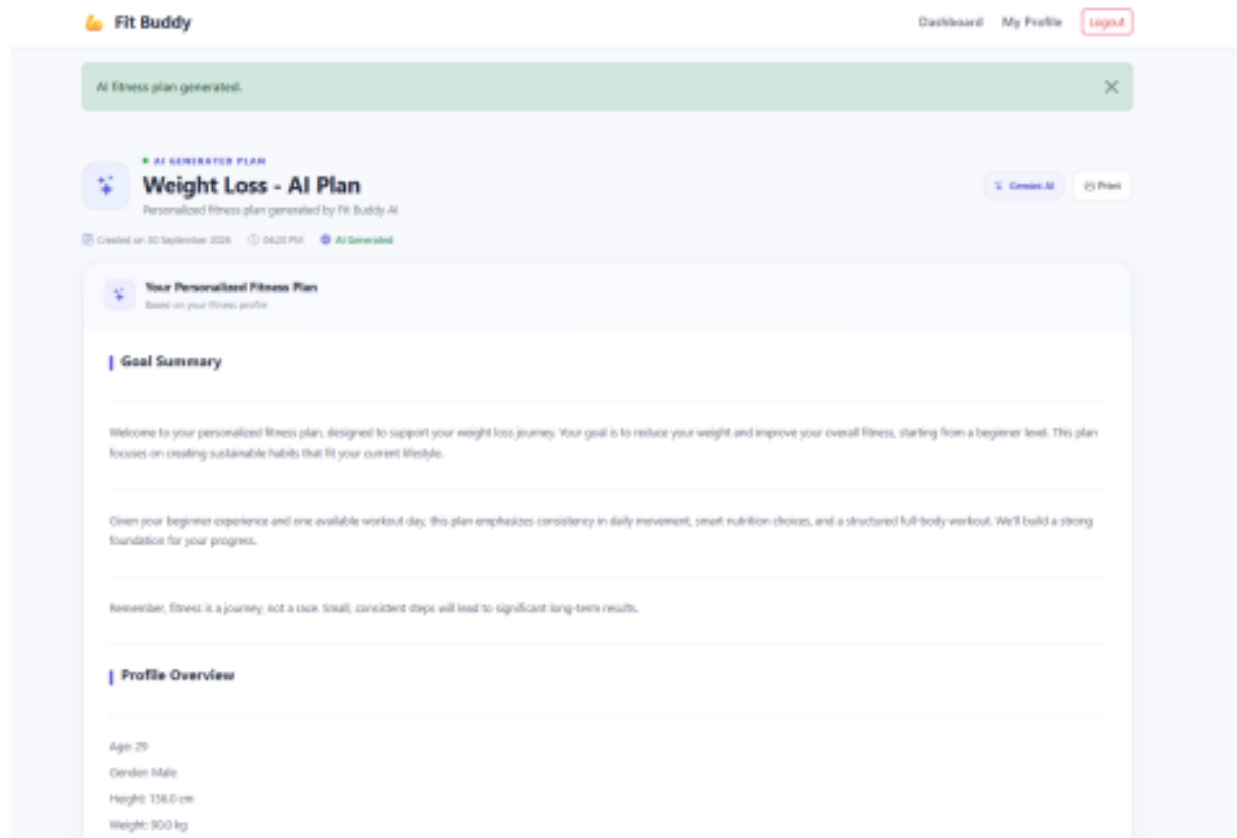


Figure 6. AI Generated Fitness Plan

Figure 7. Seven-Day Workout Schedule

Weekly workout schedule with daily workout focus, exercises, duration, and recovery guidance.

7-Day Workout Schedule	
Monday	
Workout focus: Active Recovery & Daily Movement Exercises: Aim for 30-45 minutes of walking. Break it into shorter segments if needed, like a 15-minute walk in the morning and another in the evening. Sets, repetitions or duration: Walk at a comfortable pace, gradually increasing speed or duration as you feel stronger.	
Tuesday	
Workout focus: Active Recovery & Daily Movement Exercises: Aim for 30-45 minutes of walking. Consider exploring new routes or doing light stretching at home for 10 minutes. Sets, repetitions or duration: Focus on consistent movement throughout the day.	
Wednesday	
Workout focus: Full Body Strength & Cardio (No Equipment) Exercises: Bodyweight Squats: 3 sets of 10-12 repetitions Pushups (on knees or against a wall if needed): 3 sets of 8-10 repetitions Plank: 3 sets, hold for 30-60 seconds Glide bridges: 3 sets of 12-15 repetitions Walking Lunges: 3 sets of 8-10 repetitions per leg Jumping jacks or step-jacks if available: 3 sets of 30 seconds	

Figure 7. Seven-Day Workout Schedule

Figure 8. Fit Buddy AI Chat

Interactive AI chat area where users can ask questions about the current fitness plan.

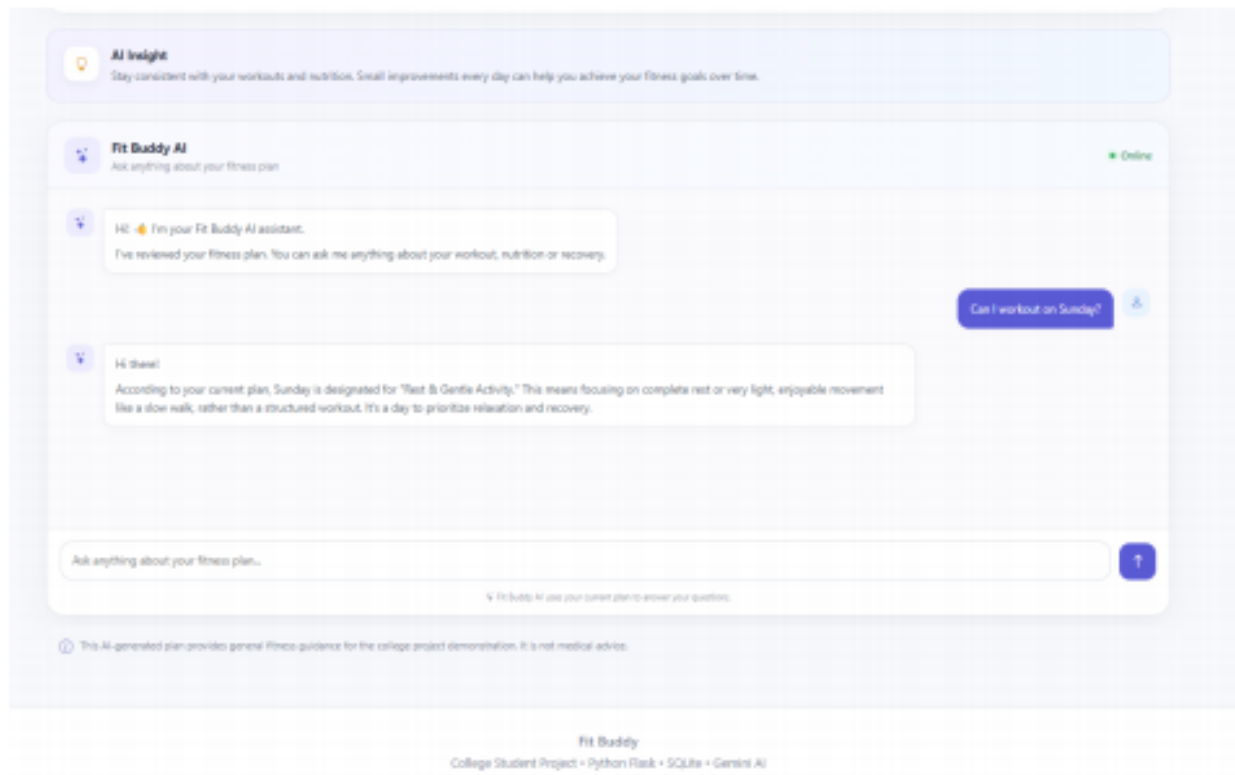


Figure 8. Fit Buddy AI Chat

Figure 9. Gemini Authentication Error

Development-time Gemini API authentication error observed during testing.

google.genai.errors.ClientError

```
google.genai.errors.ClientError: 401 (https://ai.google.dev/gemini-api/docs/api-key): {"error": {"code": 401, "message": "Request had invalid authentication credentials. Expected OAuth 2 access token, login cookie or other valid authentication credential. See https://developers.google.com/identity/sign-in/web/devconsole-project.", "status": "UNAUTHENTICATED", "details": [{"@type": "type.googleapis.com/google.rpc.errorinfo", "reason": "UNAUTHENTICATED", "metadata": {"@type": "google.protobuf.Struct", "fields": {"service": "generativelanguage.googleapis.com"}}]}}
```

Traceback (most recent call last):

```
File "C:\Users\user\Desktop\fitbuddy\venv\scripts\python.exe", line 128, in __call__
    return self._app.run_with_request(request)
File "C:\Users\user\Desktop\fitbuddy\venv\scripts\python.exe", line 154, in wsgi_app
    response = self._app.handle_request(request)
File "C:\Users\user\Desktop\fitbuddy\venv\scripts\python.exe", line 154, in wsgi_app
    response = self._app.handle_request(request)
```

Figure 9. Gemini Authentication Error

Figure 10. Project Structure

VS Code project structure showing the Flask application, services, templates, static CSS, SQLite instance, environment files, and run script.

> .venv

▼ app

> __pycache__

▼ services

> __pycache__

🔗 gemini_service.py

▼ static\css

style.css

▼ templates

↔ admin.html

↔ base.html

↔ dashboard.html

↔ home.html

↔ login.html

↔ plan.html

↔ profile.html

↔ register.html

🔗 __init__.py

🔗 auth.py

🔗 calculator.py

🔗 extensions.py

Figure 10. Project Structure

PROJECT RUN INSTRUCTIONS

13. Create and activate a Python virtual environment.
14. Install dependencies using: `pip install -r requirements.txt`
15. Create a .env file and configure SECRET_KEY and GEMINI_API_KEY.
16. Start the application using: `python run.py`
17. Open the local Flask URL shown in the terminal.

FINAL PROJECT SUMMARY

Project Name: Fit Buddy – AI Fitness Plan Generator Using Gemini Models

Backend: Python Flask

Database: SQLite

AI: Google Gemini

Frontend: HTML, CSS

Main Features: Authentication, Fitness Profile, BMI/BMR/Maintenance Calculation, AI Plan Generation, Plan History, AI Plan Chat, Responsive UI.